

# Session Four



## L'économie de tokens

Réduire le coût de l'IA des deux côtés – RTK & Caveman

# Avant de commencer



Une session pour apprendre, pas un argumentaire commercial



## Pourquoi ce sujet ?

- Tokens = argent + latence + contexte perdu
- Deux outils gratuits, vraies économies



## Discussion ouverte

- Questions bienvenues
- Partagez vos propres setups

Objectif : dépenser moins de tokens, garder la qualité

# Récap : notre parcours

## Session 1

### Du chatbot à l'agent

- IA consciente du contexte
- Même prompt, résultats différents

## Session 2

### Prompt Engineering

- Les 5 piliers
- Les anti-patterns

## Session 3

### Au-delà du contexte

- Limites de la fenêtre de contexte
- OpenCode, MCP, sous-agents

La session 3 demandait : **comment faire rentrer plus dans le contexte ?**

La session 4 demande : **comment dépenser moins de tokens dès le départ ?**

# Le programme du jour



## Partie 1 : L'économie de tokens

~10 min

Pourquoi les tokens coûtent, entrée vs sortie



## Partie 2 : RTK – couper l'entrée

~12 min

Compresser la sortie des commandes avant le contexte



## Partie 3 : Caveman – couper la sortie

~10 min

Faire parler le modèle moins, pas réfléchir moins



## Partie 4 : Combiner & décider

~5 min

Quand utiliser quoi, démo live, plan d'action

# Partie 1



## L'économie de tokens

Où partent vraiment les tokens

# C'est quoi un token, au juste ?

Un token  $\approx$   $\frac{3}{4}$  de mot

- "context" → 1 token
- "re-rendering" → 3 tokens
- Code, logs, JSON → gros en tokens

Chaque tour paie TOUT le contexte

- Le modèle relit tout à chaque message
- Un contexte de 50k tokens coûte 50k tokens... à chaque tour

Deux types de tokens, deux prix

|               | Quoi                               | Coût            |
|---------------|------------------------------------|-----------------|
| <b>Entrée</b> | Fichiers, logs, historique envoyés | moins cher      |
| <b>Sortie</b> | Ce que le modèle écrit en retour   | ~4-5× plus cher |

La taxe cachée : le gonflement du contexte

- Plus de tokens = réponses plus lentes
- Plus de tokens = débordement de contexte plus tôt

# Où partent les tokens ?

Une session de code ~30 min ≈ 118 000 tokens



## Côté entrée (ce que tu donnes)

- `cat`, `read` d'un fichier → fichier entier dans le contexte
- Sortie de `npm test` → 200 lignes de bruit
- `git diff`, `git log`, `ls -R`
- Erreurs de build, stack traces

Surtout du boilerplate répétitif



## Côté sortie (le modèle répond)

- « Bien sûr ! Je serais ravi de vous aider... »
- Ré-expliquer ce qu'il vient de faire
- Politesses, formules de précaution
- Longs résumés redits

Surtout du remplissage autour de la substance

La plupart de ces 118k tokens sont du **bruit, pas du signal**.

# Deux leviers, deux outils



RTK

Coupe l'ENTRÉE

Comprime la sortie des commandes *avant* qu'elle atteigne le modèle.

–60 à –90 % sur les commandes dev



Caveman

Coupe la SORTIE

Fait répondre le modèle de façon télégraphique, sans remplissage.

~–65 % sur les réponses

Ils visent des tokens **différents** → cumulables.



## Partie 2



### RTK – Rust Token Killer

Compresser l'entrée avant qu'elle coûte

# RTK : c'est quoi

## Un proxy CLI pour la sortie des commandes

- Un seul binaire Rust, **<10ms** d'overhead
- **100+** commandes supportées
- Se place entre ton shell et l'IA
- Apache 2.0, télémétrie désactivée par défaut

## L'idée en une ligne

La sortie brute des commandes est pleine de bruit. RTK enlève le bruit, garde le signal, *puis* l'IA lit.

```
git · ls · cat · grep · tests · docker · kubectl · aws
```

Compatible 14 outils IA : Claude Code, Copilot, Cursor, Gemini CLI, Windsurf, Cline...

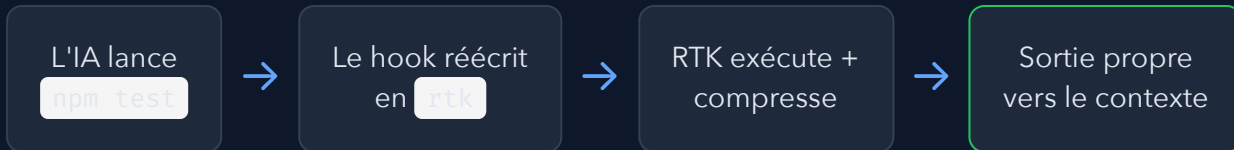
# RTK : comment ça marche

## 4 stratégies de compression

1. **Filtrage intelligent** – enlève commentaires, espaces, boilerplate
2. **Regroupement** – fichiers par dossier, erreurs par type
3. **Troncature** – garde le contexte utile, coupe la redondance
4. **Déduplication** – fusionne les lignes répétées avec un compteur

## Hook de réécriture auto

- `git status` → `rtk git status`
- Transparent, **zéro overhead de tokens**
- L'IA ne tape jamais `rtk` elle-même
- Natif Linux/macOS (Windows via WSL)



# RTK : le hook, concrètement

**Un hook = un point d'interception** L'agent (Claude Code, Copilot...) permet de brancher un script **avant** ou **après** une action.

RTK se branche sur `PreToolUse` de l'outil **Bash** : il intercepte chaque commande shell **avant exécution** et la réécrit.

**Pourquoi « zéro overhead de tokens »** La réécriture se passe **en dehors du modèle**. L'IA tape `git status` ; le hook le transforme en `rtk git status` sans que le modèle le voie. Aucun token dépensé pour ça.

La config (Claude Code – `settings.json`)

```
"PreToolUse": [  
  {  
    "matcher": "Bash",  
    "hooks": [  
      { "type": "command",  
        "command": "rtk hook claude" }  
    ]  
  }  
]
```

## Le flux du hook

1. IA veut lancer `git status`
2. Hook reçoit la commande (JSON sur stdin)
3. Renvoie `rtk git status` + `permissionDecision: allow`

# RTK : avant / après

Sans RTK – `git push`

```
Enumerating objects: 5, done.  
Counting objects: 100% (5/5), done.  
Delta compression using up to 8 threads ...  
Compressing objects: 100% (3/3), done.  
Writing objects: 100% (3/3), 312 bytes  
Total 3 (delta 2), reused 0 (delta 0)  
To github.com:org/repo.git  
a1b2c3d..e4f5g6h main → main
```

~200 tokens

Avec RTK

```
ok main
```

~10 tokens

La même info utile à l'IA :

**push réussi, branche main.**

Les 190 autres tokens étaient du bruit.

# RTK : les chiffres

Projet moyen, une session

| Opération                                       | Fréq | Standard | RTK     | Économie |
|-------------------------------------------------|------|----------|---------|----------|
| <code>ls</code> / <code>tree</code>             | 10×  | 2 000    | 400     | −80 %    |
| <code>cat</code> / <code>read</code>            | 20×  | 40 000   | 12 000  | −70 %    |
| <code>npm test</code> / <code>cargo test</code> | 5×   | 25 000   | 2 500   | −90 %    |
| Total                                           | —    | ~118 000 | ~23 900 | −80 %    |

−80 %

en moyenne sur une session

−90 %

runners de tests (le plus de bruit)

<10ms

overhead par commande

# RTK : installer & tester

## Installation

```
brew install rtk
# ou
curl -fsSL https://raw.githubusercontent.com/\
rtk-ai/rtk/master/install.sh | sh

rtk --version
```

## Commandes utiles

```
rtk gain           # analytics des économies
rtk gain --history  # historique par commande
rtk discover        # opportunités manquées
rtk proxy <cmd>     # brut, sans filtrage
```

## À savoir / limites

- Le hook de réécriture natif est Linux/macOS uniquement (Windows via WSL)
- Collision de nom : un autre `rtk` (Rust Type Kit) existe – vérifie que `rtk gain` marche
- La compression est avec perte par design – utilise `rtk proxy` quand tu veux la sortie brute

# Partie 3



Faire parler le modèle moins, pas réfléchir moins



# Caveman : c'est quoi

## Un skill qui compresse la SORTIE du modèle

- Injecte un style de communication télégraphique
- Enlève remplissage, politesses, précautions
- Garde chaque fait technique
- Claude Code + 30 autres agents

## La phrase qui compte

« Caveman affecte seulement les tokens de sortie.  
Caveman pas rendre cerveau plus petit. Caveman rendre  
**bouche** plus petite. »

**Le raisonnement intact – seule la réponse rétrécit**

Installe un fichier flag → télégraphique dès le 1er message, sans `/caveman` manuel.

# Caveman : avant / après

## Claude normal

« La raison pour laquelle ton composant React se re-rend est probablement que tu crées une nouvelle référence d'objet à chaque cycle de rendu. Quand tu passes un objet inline en prop, une nouvelle référence est créée à chaque fois, ce qui fait re-rendre l'enfant... »

~69 tokens

## Claude Caveman

« New object ref each render. Inline object prop = new ref = re-render. Wrap in `useMemo` . »

~19 tokens

Même correctif. Même raisonnement.  
**Juste sans le remplissage autour.**

# Caveman : niveaux d'intensité

**lite** – enlève seulement le remplissage

Se lit encore naturellement. Défaut sûr pour le travail partagé.

**full** – caveman classique (*défaut*)

Enlève les articles, fragments OK. ~65 % de réduction.

**ultra** – télégraphique

Compression maximale qui reste de l'anglais.

**wenyan** – chinois classique

Le plus compressé. Nouveauté / cas extrêmes.

**Sécurité intégrée : auto-clarté** Caveman désactive la compression pour les avertissements de sécurité, les confirmations d'actions irréversibles, et les séquences multi-étapes où enlever des mots risque un malentendu. Code, commits, PRs restent en prose normale.

# Caveman : les chiffres

–65 %

réduction moyenne de la sortie

22–87 %

fourchette selon la tâche

~30s

installation (Node ≥18)

| Tâche                          | Avant | Après | Économie |
|--------------------------------|-------|-------|----------|
| Explication re-render React    | 1 180 | 159   | –87 %    |
| Correctif auth middleware      | 704   | 121   | –83 %    |
| Setup connection pool Postgres | 2 347 | 380   | –84 %    |

Installation : `curl -fsSL https://raw.githubusercontent.com/JuliusBrussee/caveman/main/install.sh | bash`

# Partie 4



## Combiner & décider

Les cumuler, et connaître les compromis

# RTK vs Caveman – pas vs, mais ET

|                  | RTK                                    | Caveman                         |
|------------------|----------------------------------------|---------------------------------|
| Cible            | Tokens d'entrée                        | Tokens de sortie                |
| Quoi             | Sortie des commandes                   | Réponse du modèle               |
| Comment          | Proxy CLI + hook                       | Skill de style dans le contexte |
| Forme            | Binaire Rust                           | Prompt / skill                  |
| Économie         | −60 à −90 % sur les cmds               | ~−65 % sur les réponses         |
| Impact qualité ? | Sortie avec perte, pas le raisonnement | Raisonnement intact             |
| Idéal pour       | Travail riche en tests/build/git       | Longues sessions bavardes       |

Tokens différents → **lance les deux, les économies se cumulent.**

# Quand utiliser quoi

## ✓ Prends RTK quand

- Beaucoup de runs tests / build / lint
- Lecture de gros fichiers ou logs
- Revue lourde en `git diff` / `git log`
- Sorties Docker / kubectl / AWS
- Le contexte se remplit trop vite

## ✓ Prends Caveman quand

- Longues sessions d'allers-retours
- Tu veux des réponses, pas des dissertations
- Coût/latence comptent pour toi
- Travail solo ou expert

## ⚠ Attention quand

- **Onboarding / pédagogie** → les réponses verbeuses aident ; Caveman `true` ou off
- **Transcripts partagés** → une sortie télégraphique peut perdre les autres
- **Tu as besoin du brut** → `rtk proxy` pour contourner la compression
- **Compliance / audit** → sache que RTK enlève des données ; vérifie que rien de critique n'est perdu

# Démo live



## Démo RTK

1. `rtk gain` – économies actuelles
2. Une suite de tests brut vs `rtk` – comparer le nombre de lignes
3. `git push` avant/après
4. `rtk discover` sur l'historique réel



## Démo Caveman

1. Poser une question de debug en mode normal
2. `/caveman full`, reposer
3. Comparer les compteurs de tokens côte à côte
4. Montrer l'auto-clarté sur un prompt « delete table »

Bonus : les deux ensemble sur une vraie tâche – mesurer la baisse combinée.



# Les limites, honnêtement

## RTK

- Avec perte par design – peut cacher un détail dont tu avais besoin
- Un outil de plus à installer/maintenir dans la chaîne
- Le hook natif exclut Windows classique
- Vérifie que tu as le bon binaire `rtk`

## Caveman

- Sortie télégraphique moins lisible pour les nouveaux
- Style seulement – ne corrige pas un mauvais prompt
- Tokens de sortie seulement – l'entrée intacte
- L'équipe peut ne pas aimer le ton « caveman »

Les deux sont **gratuits, opt-in, réversibles**. Teste sur une vraie tâche, mesure, décide.

# À retenir



**Les tokens ont deux côtés** L'entrée (ce que tu fournis) et la sortie (ce qu'il dit). Coupe les deux.



**RTK tue le bruit d'entrée** -80 % sur une session, <10ms de coût, transparent.



**Caveman tue le remplissage de sortie** ~-65 % sur les réponses, raisonnement intact.



**Ils se cumulent** Tokens différents. Lance les deux. Mesure avec `rtk gain`.

# Plan d'action

❑ Installer RTK, lancer `rtk discover` sur ton historique

❑ Tester Caveman `lute` une semaine, regarder les économies en statusline

❑ Lancer les deux sur un vrai ticket, rapporter la baisse combinée

❑ Décider un défaut d'équipe : quand télégraphique, quand verbeux

[github.com/rtk-ai/rtk](https://github.com/rtk-ai/rtk)

[github.com/juliusbrussee/caveman](https://github.com/juliusbrussee/caveman)

# Questions ?



Dépenser moins de tokens. Garder la qualité.

Session Four – L'économie de tokens